

# Building AuditMind: An Agentic RAG Pipeline for Zero-Hallucination Audit Analysis

---

Financial and statutory audit documents, Independent Auditor's Reports, CARO annexures, Board Reports, are dense, tabular, and unforgiving. An auditor checking whether a physical inventory discrepancy crossed a 10% threshold, or verifying a specific statutory clause, cannot afford an AI system that hallucinates a number or quietly summarizes away a required list item.

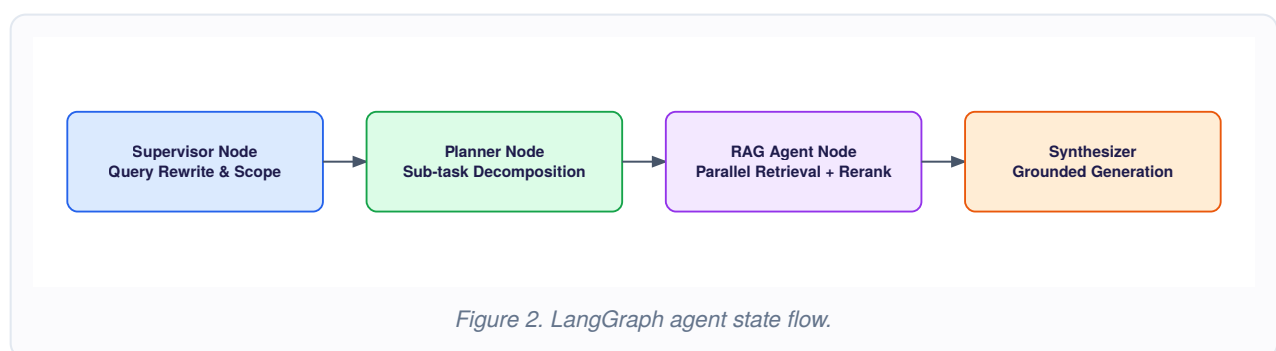
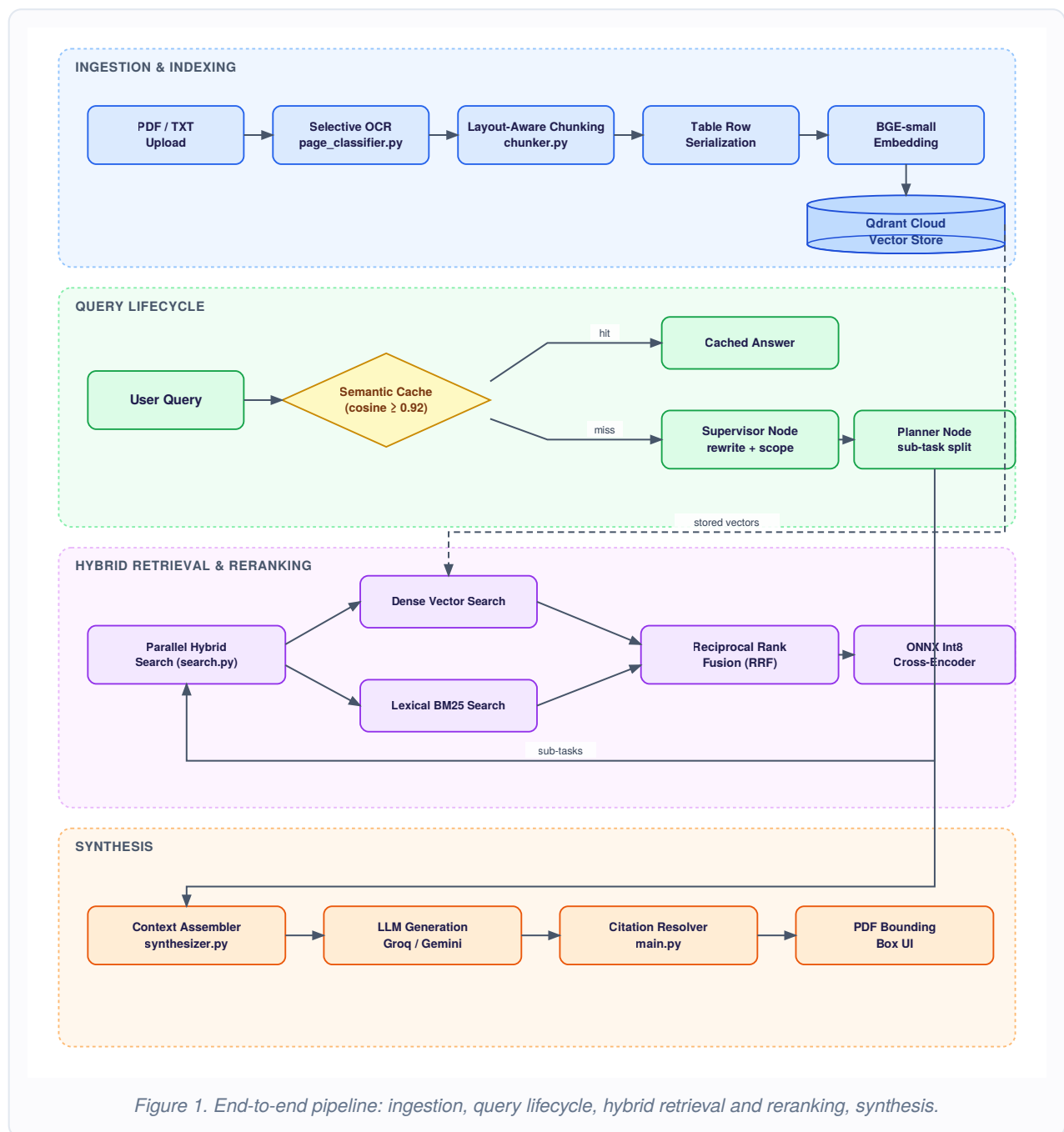
Plain LLM chat over PDFs fails here for exactly that reason: LLMs confidently invent figures and compress exhaustive statutory lists into vague summaries. Naive RAG pipelines fail differently. They flatten tables into noise, conflate adjacent enumerated paragraphs, and retrieve semantically similar but legally wrong clauses, returning a generic fraud statement when the question asked about CARO Clause (xi) specifically.

AuditMind is my attempt to close that gap: an agentic, layout-aware, hybrid-retrieval RAG system that enforces strict grounding and gives every answer a verifiable, clickable citation back to the exact bounding box on the source PDF.

## What it actually does

Given a large audit PDF, AuditMind chunks it while preserving table structure and page layout, indexes it with hybrid dense and lexical search, and answers natural-language questions with a LangGraph-orchestrated agent. Every claim in the answer is traceable to a highlighted region of the original document, so there's nothing to take on faith.

## Architecture



A document is selectively OCR'd, then chunked so tables and prose are handled differently. Chunks are embedded and pushed into Qdrant. A query first checks a semantic cache, then passes through a Supervisor node (rewrite, scope resolution) and a Planner node that splits compound questions into parallel sub-tasks. Each sub-task runs hybrid search (dense + BM25, merged with Reciprocal Rank Fusion), gets reranked by an Int8-quantized cross-encoder, and the top candidates are assembled into a strict grounding prompt. The LLM generates an answer with hidden UUID citations, which get resolved into clickable markdown links mapped to exact PDF bounding boxes.

## The engineering journey

**Pure vector search wasn't enough.** Early on, dense embeddings alone kept returning semantically-similar-but-wrong clauses, a query for "Rule 11" would surface a generically similar compliance paragraph instead of the actual rule. Adding BM25 lexical search and merging both rankings with Reciprocal Rank Fusion fixed precision on exact statutory terms without losing semantic flexibility.

**Tables were invisible.** Raw markdown tables produce near-zero-variance embeddings, so structured data like CARO compliance tables just never got retrieved. The fix was serializing every table row into a natural-language sentence before embedding, so a row becomes something a dense model can actually distinguish from another row.

**Then tables broke differently.** Auditor signature blocks and certain financial matrices are pivoted, entities as columns instead of rows. Row-by-row serialization merged unrelated partners into nonsense sentences. I added column-detection logic that spots pivoted layouts by proper-noun density in headers and switches to per-entity-column serialization instead.

**Identical queries gave different answers.** Regression testing showed non-deterministic retrieval on repeated identical queries. The cause was an uncapped Qdrant fetch with no tie-breaker on equal scores. Switching to a scroll-based fetch with a strict `(score, id)` tie-breaker fixed retrieval-layer determinism. A smaller residual non-determinism remains at the LLM inference layer itself (Groq's backend), which I accepted rather than chased, since strict prompt grounding keeps the variance stylistic, not factual.

**Latency blew past budget.** Complex queries were hitting 8 to 12 seconds against a 3-second target. Quantizing the cross-encoder to Int8 ONNX cut reranking time substantially. Disabling Gemini's automatic function-calling removed unnecessary round-trips. Skipping the LLM query-rewrite step when no conversational pronouns were present in the query removed another unneeded LLM call. Together these brought latency back under budget.

**Caching had to be trustworthy, not just fast.** A semantic cache backed by Qdrant crushes latency on repeated questions, but a naive cache risks permanently serving a bad answer. I scoped it to a high 0.92 cosine threshold and explicitly excluded fallback/error responses from being cached, with entries only persisted after implicit confirmation that the answer was good.

**A bracket bug broke every citation.** Switching to Groq for synthesis speed silently broke citation rendering. Some Groq responses emitted Unicode fullwidth brackets instead of ASCII ones, which

slipped past the citation regex entirely. A normalization pass mapping known Unicode bracket variants to ASCII before regex resolution fixed it.

**The reranker couldn't be fully trusted for critical sections.** Key Audit Matters and auditor signature blocks would occasionally get down-ranked by the cross-encoder even though they're non-negotiable for compliance checks. I added structural entity guarding that force-promotes these blocks to the top of the candidate pool when the query implies it, overriding the reranker.

## Trade-offs

- **40 candidates, not 100.** Capping the merged candidate pool at 40 keeps the LLM context tight and avoids "lost in the middle" degradation, at the cost of recall on very large tables.
- **CPU-only ONNX reranker over GPU.** Chosen for cheap, portable deployment on standard PaaS platforms, trading peak throughput for cost and simplicity.
- **Accepted LLM-level non-determinism.** Rather than switching providers for strict determinism, I kept Groq's speed and relied on strict grounding prompts to bound the variance to phrasing, not facts.
- **1800-character reranker window.** Keeps memory bounded during Int8 matrix multiplication, at the risk of truncating a sentence exactly at a window boundary.

## Current limitations

- Scanned-page detection is a simple character-count heuristic; complex mixed image/text pages can slip past OCR undetected.
- Very large tables (20+ pages) can't fit entirely into the LLM's context window for full cross-table summarization.
- The ONNX reranker runs on CPU with a hardcoded thread count, so it degrades under concurrent load.
- Equity-accounted Joint Ventures and Associates aren't structurally line-itemized in these reports, so the system leans on a prompt-level patch rather than true structural understanding.
- The deployment currently excludes the ONNX model from the build artifact, forcing a slower PyTorch fallback at runtime, and the process manager runs without explicit timeouts.

## What's next

1. Fix the deployment pipeline so the quantized ONNX model ships in the container and worker timeouts are explicit.
2. Replace the character-count OCR heuristic with a vision-language model fallback for mixed-content pages.

3. Route large tabular aggregation queries to a dataframe execution layer instead of forcing them through the LLM context window.
4. Move the cross-encoder to an independent, scalable worker pool to remove the CPU contention bottleneck.

AuditMind is still evolving, but the core thesis holds: grounded, traceable answers over structured documents require more than a vector store and a prompt. They require an architecture that respects the shape of the data.